ESP32 ファームウェア書き込みガイド

DUMMY INTERCOM・2×ESP32 構成(#1 受信側 / #2 送信側)の BUILD & FLASH

項目	内容
対象ボード	ESP32-WROOM-32 devkit ×2(4MB フラッシュ、USB-シリアル CP2102 等)
ターゲット	`esp32` (無印 ESP32。BR/EDR Classic 必須。S2/S3/C3/C6/H2 不可)
SDK	ESP-IDF v5.4 + Bluedroid
プロジェクト	`/home/subcon/SOURCE_CODE/dummycom2`

0. 2 台のファームウェアは別物

基板	役割	ビルド元	BT 名	コマンドの形
#1	受信側:SRL3 メッシュ会話を受信し I2S(マスタ)で出力	リポジトリ直下 `main/`	DummyIntercom	`idf.py …`
#2	送信側:I2S(スレーブ)受信し、スマホ/Insta360 X4 へ無線マイク	`hf_unit/`	DummyIntercomMic	`idf.py -C hf_unit …`

> **#1 と #2 は必ずそれぞれに書き込む**(同じバイナリでは動きません)。書き込みは 1 枚ずつ USB につないで行います。

1. 必要なもの

- ホスト PC(Linux。Debian 13 前提)/ データ通信対応の USB ケーブル(充電専用不可)
- ESP32 devkit ×2 / ジャンパ線(配線は別紙 `esp32-esp32-wiring.pdf`)
- ESP-IDF v5.4(このマシンは `~/esp/esp-idf` 、ツール `~/.espressif` に導入済み。再導入は `bash tools/setup_espidf.sh`)
- ユーザーが `dialout` グループ所属(未所属なら `sudo usermod -aG dialout $USER` 後に再ログイン)。ModemManager が邪魔なら `sudo systemctl mask ModemManager`

2. 環境有効化(シェルを開くたび)

```
export IDF_TOOLS_PATH=$HOME/.espressif
. $HOME/esp/esp-idf/export.sh
idf.py --version        # => ESP-IDF v5.4
```

3. ESP-IDF パッチの適用(初回のみ)

ローカルの ESP-IDF に当てるパッチです(ESP-IDF を更新・再取得すると消えるので再適用)。

パッチ	必要な基板	目的
`esp-idf-v5.4-hfp-client-no-post-slc-at.patch`	**#1(HF 役)必須**	SLC 確立 ~30 秒後の `AT response timeout` 切断を防ぐ
`esp-idf-v5.4-hfp-bac-lc3.patch`	#1 を AG 役でビルドする場合のみ	mSBC(16kHz)ネゴシエーション

```
cd ~/esp/esp-idf
P=/home/subcon/SOURCE_CODE/dummycom2/patches/esp-idf-v5.4-hfp-client-no-post-slc-at.patch

# 1) 適用済みか確認(逆方向チェックが通れば「適用済み」)
git apply --check -R $P && echo "適用済み(何もしなくてよい)"

# 2) 未適用の場合のみ適用
git apply $P
```

`patch does not apply` が出た場合は、まず適用済みの可能性が高い(上の 1 を実行)。1 も通らない場合は ESP-IDF が v5.4 でない可能性があるので `git describe --tags` で確認。

> パッチは共有の ESP-IDF 側に当たるため、#2(`hf_unit`、同じ HFP クライアント実装を使用)のビルドにも反映されます。#2 でスマホ/X4 との動作に影響するかは実機未確認です。異常があれば `git checkout components/bt/host/bluedroid/bta/hf_client` 相当で戻して切り分けてください(詳細は `patches/README.md`)。

4. #1(受信側)のビルドと書き込み

4-1. ビルド

```
cd /home/subcon/SOURCE_CODE/dummycom2
idf.py set-target esp32     # 初回、またはターゲット変更時のみ
idf.py build
```

設定は `main/app_config.h` にあります。既定値:`BT_ROLE = BT_ROLE_HF`、`AUDIO_OUT_MODE` = I2S16K(**PCM5102A 不要・I2S で #2 へ**)。
従来の PCM5102A/3.5mm 出力に戻すときは `AUDIO_OUT_MODE` を 0 側(DAC48K)に変更して再ビルドします。

4-2. 書き込み

```
# #1 だけを USB に接続し、ポートを確認
ls /dev/ttyUSB*          # 例: /dev/ttyUSB0
idf.py -p /dev/ttyUSB0 -b 115200 flash monitor
```

モニタ終了は **Ctrl +]**(ログは 115200 bps)。書き込みは **`-b 115200` を付ける**(既定の 460800 では `No serial data received` で失敗することがあるため)。

4-3. 起動時に出るログ(#1)

ログ	意味
`app_main: own address: …`	BT 起動・自機アドレス
`audio_i2s: I2S0 TX up: 16000 Hz/16-bit mono, BCLK=26 WS=25 DOUT=27`	I2S マスタ出力開始(I2S16K 時)
`bt_hf_client: HFP-HF ready, name="DummyIntercom"`	ペアリング待ち受け
`SLC connected` → `mSBC/16kHz SCO connected` または `CVSD/8kHz SCO connected`	SRL3 と接続・音声リンク確立

5. #2(送信側)のビルドと書き込み

> **書き込み前に必ず**:#1 から #2 へ渡している **5V 線(VIN)を外す**。#2 に USB を挿したまま 5V 線を接続すると電源同士が逆流します。

5-1. ビルド

```
cd /home/subcon/SOURCE_CODE/dummycom2
idf.py -C hf_unit set-target esp32     # 初回のみ
idf.py -C hf_unit build
```

設定は `hf_unit/main/app_config.h`。既定 `AUDIO_IN_MODE` = I2S(スレーブ受信)。従来のアナログ ADC(GPIO34)入力に戻すときは ADC 側に変更して再ビルド。

5-2. 書き込み

```
# #1 を外し、#2 だけを USB に接続(5V 線は外したまま)
ls /dev/ttyUSB*
idf.py -C hf_unit -p /dev/ttyUSB0 -b 115200 flash monitor
```

> 2 枚を同時に PC につなぐと `ttyUSB0` / `ttyUSB1` になり、順序が入れ替わることがあります。`dmesg | tail` や、挿した直後に増えた ttyUSB を確認し、**ログ(BT 名)で #1 / #2 を必ず見分けて**ください。

5-3. 起動時に出るログ(#2)

ログ	意味
`i2s_capture: I2S0 RX slave up: 16000 Hz/16-bit mono, BCLK=26 WS=25 DIN=27`	I2S スレーブ受信開始
`HFP-HF ready, name="DummyIntercomMic"`	スマホ/X4 からのペアリング待ち受け
`I2S clock recovered`	#1 からの BCLK を検出(#1 未接続/未給電だと出ない)

6. 書き込み後の運用手順

1. 両基板の書き込み完了後、USB を外す。
2. 配線(I2S 3 本 + GND + 5V)を確認し、**5V 線を接続**(配線図参照)。
3. **#1 の USB だけ**に給電する(#2 は #1 の VIN 経由で起動)。
4. #2(`DummyIntercomMic`)をスマホ/Insta360 X4 とペアリング(マイクとして選択)。
5. #1(`DummyIntercom`)を SRL3 のユニバーサルインターコム・ペアリングで接続(PIN `0000`)。

ボンディング消去:各基板の BOND-CLR(GPIO14)を 3 秒長押し。

LED(各基板とも)

LED	消灯	点滅	点灯
PWR(GPIO32)	-	5Hz = BT 起動失敗	起動完了
LINK(GPIO33)	未接続	1Hz = ページング/再接続	ACL/SLC 確立
SCO(GPIO13)	音声リンクなし	約 10Hz = バッファ異常	SCO 確立中

7. トラブルシューティング

症状	対処
`idf.py: command not found`	そのシェルで §2 を実行していない
`-C hf_unit` を付け忘れた	#1 のバイナリが #2 に書かれる。BT 名が `DummyIntercom` になっていたら #2 を書き直す
`Permission denied: '/dev/ttyUSB0'`	dialout 追加後に再ログイン(応急: `sudo chmod a+rw /dev/ttyUSB0`)
`Unable to verify flash chip connection (No serial data received.)` (`Changing baud rate to 460800` の直後)	ボーレート過大。`-b 115200` を付けて再実行。改善しなければ `-b 57600`、短い良質なデータケーブル、USB ハブを使わない、書き込み中は他基板・配線を外して単体にする、書き込み前に `erase-flash`
`Failed to connect to ESP32`	データ対応ケーブルか確認。BOOT(IO0)を押したまま EN を一度押して離し、BOOT を離してから再実行
`/dev/ttyUSB*` が出ない	ケーブル/ポート交換、`dmesg | tail`、ModemManager を mask
#1 が SLC の約 30 秒後に切断(`AT response timeout`)	§3 のパッチ未適用。適用して #1 を再ビルド・再書き込み
#2 に `I2S clock recovered` が出ない	#1 未起動、BCLK(GPIO26)/WS(25)/DATA(27)/GND の配線、#1 が `AUDIO_OUT_MODE`=I2S16K か確認
brownout / 起動ループ	USB 電源が 5V/1A 未満。#2 の VIN–GND 間に 100〜470µF を追加
ビルドエラー(設定切替後)	`rm -rf build sdkconfig && idf.py set-target esp32 && idf.py build`(#2 は `hf_unit/build`, `hf_unit/sdkconfig`)
実機の音が出ない・左右ずれ	I2S のスロット(左)が合っているか。両側とも `app_config.h` の設定を確認

8. コマンド早見表

```
# 環境有効化(毎シェル)
export IDF_TOOLS_PATH=$HOME/.espressif; . $HOME/esp/esp-idf/export.sh
cd /home/subcon/SOURCE_CODE/dummycom2

# ---- #1 受信側 (main/) ----
idf.py build
idf.py -p /dev/ttyUSB0 -b 115200 flash monitor

# ---- #2 送信側 (hf_unit/) ※5V 線を外して ----
idf.py -C hf_unit build
idf.py -C hf_unit -p /dev/ttyUSB0 -b 115200 flash monitor

# 共通
idf.py [-C hf_unit] fullclean                    # build/ を消す
idf.py [-C hf_unit] -p PORT -b 115200 erase-flash        # フラッシュ全消去(ボンドも消える)
idf.py [-C hf_unit] size                         # サイズ内訳
```

参照:README.md / hf_unit/README.md / patches/README.md / docs/wiring_dual_esp32.md / esp32-esp32-wiring.pdf。ログ文字列は main/ と hf_unit/main/ の実装に準拠。実機での動作は未確認です。